

嵌入式系统架构设计

通用嵌入式系统架构搭建指导 —— 分层 / 解耦 / 可移植 / 可扩展

一套面向各类嵌入式系统的固件架构方法论：电机 / 运动控制器、主控板、带众多外设与交互的 UI/HMI 板、传感采集板、通信网关等。

这些系统的功能模块差异很大（有的重实时控制、有的重界面交互、有的重多外设采集），但架构思路是一致的——本文给出这套一致的思路，并以几类典型系统为例。

适用对象：架构设计者、新项目搭建者、平台化 / 可复用代码维护者。

目标：让"换芯片、换外设、加功能、上产品"都只改该改的那一层，而不牵一发而动全身。

排版说明：凡需对齐的内容都放在代码块中——代码块在记事本与渲染视图

(GitHub/VSCode)都保留空格，故任何工具打开都整齐；对齐点只用 ASCII，中文不参与列对齐（规避等宽字体下中文双宽），不用 Markdown 竖线表格。建议用等宽字体、关闭自动换行查看。



复制代码

目录

1. 设计目标与总原则
 2. 分层架构
 3. 解耦手段（六种）
 4. 可移植性
 5. 可扩展性
 6. 实时性 / 事件驱动与安全
 7. 工程规范与约定
 8. 落地检查清单
- 附．典型系统的层次映射（电机 / UI / 采集主控）

1. 设计目标与总原则

四个相互支撑的目标（左侧目标，冒号后为含义 + 达标标准）：



复制代码

Layering : 分层—职责按抽象层级切分、依赖只向下。任一层能讲清"输入/输出/依赖谁", 无反向或跨层调用

Decoupling : 解耦—模块经稳定接口交互、不依赖彼此内部。改一个模块内部实现, 不需动调用方

Portability : 可移植—换 MCU/编译器/OS 只动隔离层。核心算法与业务零硬件依赖, 换平台只重写 bsp

Extensibility : 可扩展—加外设/功能/界面/接口是"加法"而非"改法"。新增能力主要是加文件 + 改配置

贯穿这四点的主线只有一条: 用"配置 + 接口注入 + 编译期裁剪", 把"变化"从"稳定"里隔离出去。



复制代码

变化的 : 硬件引脚、外设型号、业务参数、启用哪些功能
稳定的 : 算法、状态机、协议逻辑、层间接口

架构的价值, 就是不让前者污染后者——无论这个系统是电机驱动、UI 交互还是数据采集。

2. 分层架构

2.1 层次与职责 (通用, 附各类系统举例)

用户代码分为 `manager -> application -> middleware -> bsp` 四层, 外加公共组 `common / os / config / sdk`。



复制代码

manager : 装配层—创建实例、注入依赖、注册任务、启动调度 (唯一"组装整机"的地方)

application : 业务层—本系统"做什么"的功能模块, 按业务拆子模块
例) 电机板=运动/电流控制; UI 板=界面状态机/菜单/交互; 采集板=采样调度/上报

middleware.lib : 中间件·纯算法/协议—确定性输入->输出、无硬件依赖、可单元测试
例) FOC/滤波/PID; GUI 绘制/字库/事件分发; 数据融合; 协议编解码

middleware.user : 中间件·适配件—把 lib 需要的数据用具体硬件喂入 (桥接 lib 与硬件)

bsp : 板级—外设初始化与读写, 向上出与芯片无关的 `bsp_*` 接口 (换芯片只动这层)

UART/GPIO
hal
common
可被各层调用
os
config

- : 芯片厂商外设库 / CMSIS (第三方, 不改)
- : 通用件—纯软件工具(滤波/环形缓冲/CRC/状态机/窗口监视器...),
- : 裸机/RTOS 抽象(时基、临界区、任务、事件)
- : 全部可调项(引脚、外设型号、业务参数、功能裁剪、协议地址...)

: 芯片厂商外设库 / CMSIS (第三方, 不改)

：通用件—纯软件工具(滤波/环形缓冲/CRC/状态机/窗口监视器...),

：裸机/RTOS 抽象（时基、临界区、任务、事件）

: 全部可调项 (引脚、外设型号、业务参数、功能裁剪、协议地址...)

44 45 46 47 48

2.2 分层框图

上层在上，依赖一律向下；左侧方框为纯 ASCII，右侧为该层职责（举例跨系统类型）：

 复制代码

| manager | 装配：初始化 / 依赖注入 / 任务注册 / 调度 |

application	业务：控制 / 界面交互 / 采集上报 / 安全（随系统而异，按功能拆子模块）
-------------	---

middleware	lib : 纯算法·协议 (FOC / GUI / 融合 / 编解码, 无硬件依赖, IO 由上层注入)
------------	--

user: 硬件适配器 (换算 / 桥接)

| bsp | 外设驱动: PWM/ADC/CAN/LCD/触摸/传感器/UART... (向上出
bsp_*)

| ha1 | 芯片厂商库 / CMSIS

1 依赖只向下

横切公共组（可被相应层调用）：`config(配置)` . `common(通用件)` . `os(裸机/RTOS抽象)` . `sdk(协议)`

2.3 依赖方向铁律

 复制代码

下层需要"上层的能力"时，由上层注入（见 3.1），而不是下层去找上层。

自检：厂商库符号(如 `gd32*/stm32*` 等)应只出现在 `bsp` 与 `hal`；一旦在

2.4 目录即层次

目录名 = 层名，新人不读代码就能定位；各层内部子模块随系统而异：

project/	
manager/	: 装配层（唯一"组装整机"处）
application/	: 业务层（按功能拆子模块，随系统而异）
middleware/lib/	: 纯算法 / 协议（无硬件依赖）
middleware/user/	: 硬件适配件
bsp/	: 板级外设驱动（向上出 <code>bsp_*</code> ）
hal/	: 芯片厂商库
common/	: 通用件
os/	: 裸机 / RTOS 抽象
config/	: 全部可调项
sdk/	: 对外通信协议

3. 解耦手段（六种）

每种手段 = 通用原则 + 跨系统示例（不局限于某一种系统）。

3.1 依赖注入 —— 让纯算法不碰硬件

原则：算法/业务库要"读传感器/写外设"时，不直接调 `bsp`，而是接收一组函数指针(接口结构体)，由装配层

在创建时传入。库因此编译期就与硬件无关，可脱离硬件在 PC 上单测、可整体搬到别的项目。

示例：电机反馈库接收 {读霍尔，读编码器} 由 `bsp` 注入；GUI 库接收 {画点/刷屏，读触摸} 由 `LCD/触摸`

`bsp` 注入；采集库接收 {读某路传感器} 由 `I2C/SPI bsp` 注入。库只认接口、不认芯片。

```
/* 库只声明"我需要什么"，不知道谁来实现 */
typedef struct { void (*flush)(const uint16_t* buf, int n); uint8_t
(*read_touch)(int* x,int* y); } lcd_io_t;
gui_t *gui_create(const lcd_io_t *io);
```

```
/* 装配层：把具体 bsp 实现注入 */
static const lcd_io_t io = { bsp_lcd_flush, bsp_touch_read };
s = gui_create(&io);
```

3.2 门面(Facade) —— 子系统对外只留一个头

[复制代码](#)

原则：一个子系统对外只暴露一个接口头，隐藏全部内部实现。上层只依赖门面，子系统内部随意重构。

示例：电机子系统只出 `motor.h`；显示子系统只出 `display.h`；存储子系统只出 `storage.h`。无论子系统内部

多少文件，上层只 `include` 那一个门面头，内部结构不外泄。OS 抽象同理封在 `os_port.h` 之内。

3.3 配置驱动 —— 变化集中在 config 层

[复制代码](#)

原则：一切"随产品/硬件/业务变化"的量收敛到 `config/*.h`，其余代码只读宏、绝不内嵌魔数。

示例：`board_cfg`(引脚/外设映射/标度)。`业务_cfg`(如 `motor_cfg` / `ui_cfg` / `sensor_cfg`)。

`comm_cfg`(接口/地址)。`app_cfg`(功能裁剪/OS)。`func_cfg`(各开关)。

评审时若在 `.c` 里发现引脚号、屏参、标定系数，即"配置泄漏"，应上收 `config`。

3.4 编译期裁剪 —— 不用的功能不进固件

[复制代码](#)

原则："功能开关宏 + 条件编译"让关闭的功能不参与编译，既省 `Flash/RAM` 又降耦合，开关集中在 `config`。

示例：`FEAT_XXX=0` 时对应实现整文件不编译；实例数量(如 电机数/传感器数/页面数)由宏决定静态分配。

裁剪的"减法自由"与可扩展的"加法自由"对称——同一套代码可裁成高低配。

3.5 OS 抽象 —— 裸机/RTOS 对业务透明

[复制代码](#)

原则：业务不直接调 RTOS API，而是调一层薄抽象(时基/临界区/任务或周期作业/事件)；切换只换抽象层。

示例：业务统一用 `sched_add(name, period, job)` 注册周期作业、用事件接口收异步事件；裸机后端在超循环

里轮询，RTOS 后端每作业一个任务——上层不感知。一个宏(`USE_RTOS`)切换。

3.6 检测与处理解耦 —— 事件 / 故障的统一处置



复制代码

原则：“发现”与“应对”分开：检测器只产出状态/事件/故障位、不动作；集中处理器把它们映射到响应。

加一个被监视项不牵动响应逻辑，反之亦然。

示例：电机板——过压/过温检测只置故障位，处理器按严重程度决定降额/停机/刹车/锁存；UI/采集板——传感

超限、通信超时同样只置事件位，处理器统一决定告警/重连/安全态。检测与处理各自演进。

4. 可移植性

4.1 三道隔离



复制代码

1) BSP/HAL 隔离：芯片相关只在 `bsp/` 与 `hal/`。换 MCU=重写 `bsp/*.c`(保持 `bsp_*` 接口不变)+替换厂商库

2) 纯软件库零依赖：`middleware/lib` 不含任何芯片头，IO 全靠注入(3.1)，可脱离硬件编译/单测/复用

3) 配置集中：板级/产品差异全在 `config/`，移植=换一套 `config`

4.2 数据类型与数值约定要“平台无关”



复制代码

用定长类型(`int32_t` 等)、避免浮点与平台字长假设；量纲用归一化/定点等与硬件无关的约定。

例) 控制量归一化(与具体电压/量程无关)、角度用整周整数、增益用 `Q` 格式定点——换平台无需改算法。

4.3 移植时改什么 / 不改什么

[复制代码](#)

换 MCU / 厂商库

动: bsp/*.c 重写 + hal/ 替换
/ config 业务值

不动: application / middleware

换 OS (裸机 <-> RTOS)

动: os/ 后端 + USE_RTOS 宏

不动: 业务与算法

换 编译器 (如 AC5 <-> AC6)

动: 启动文件 / 内联汇编 / 端口文件

不动: C 逻辑 (若守 C 标准)

5. 可扩展性 ("怎么加"配方)

好的架构让"加能力"变成有章可循的配方, 而不是四处改代码。每条为"key 在上、做法缩进在下"。

[复制代码](#)

加一个外设 / 驱动

在 bsp 加一个 bsp_xxx (实现统一 bsp_* 接口), config 配引脚/参数, 上层经接口调用; 算法/业务不动

加一个业务功能模块

在 application 加子模块 + 其门面头, manager 注册其任务, 用 FEAT_* 开关裁剪; 子模块间不互相引用

加一个界面 / 页面 / 菜单项 (UI 类)

在界面状态机注册一个新页面(绘制+输入处理回调), 走统一 GUI/事件接口; 不改其它页面

加一路传感器 / 执行器

bsp 加读/写接口 + config 加映射; 若走算法, 按统一接口在 middleware 实现并在工厂登记

加一个通信接口 / 一条命令

物理接口用宏在 bsp 层选择(如 CAN/UART 共用协议逻辑、仅收发字节不同); 新命令在协议适配器加一支

加一种参数来源 (手动 .h <-> 运行期 / 持久化)

"来源开关 + 统一参数块": 编译期默认值 / 运行期整定 / Flash 持久化 三路写同一份参数, 开关定优先级,

即可"零代码适配"(现场调好即落盘、重启自动加载)而不改源码

6. 实时性 / 事件驱动与安全

不同系统的时间模型不同（控制类重周期、交互类重事件），但都按"尺度/来源分层、互不阻塞、安全兜底"来落地。



复制代码

按尺度/来源分层：硬实时(如电流环)放中断；固定周期(控制环/状态机/巡检)放定时任务；事件驱动(按键/触摸/收帧/中断上报)放事件队列+处理；慢任务(界面刷新/上报/日志/喂狗)放低优先级
共享数据单点化：对外数据收敛到一个结构—快变量在 ISR 更新、慢变量在周期/事件任务更新，读取走统一门面；跨上下文的多字节量注意读写一致性(临界区/双缓冲)
安全默认与兜底：异常->确定的安全态(停机/断输出/安全界面)；看门狗独立于业务(跑飞即复位)；通信/输入丢失有超时处理；关键保护可逐项开关、阈值可整定或动态
阻塞操作约束：写 Flash / 长时外设操作会阻塞或冻结中断，须在安全时机(停机/输出关闭)进行

7. 工程规范与约定



复制代码

- 1、统一文件头模板：Copyright / File / Description / Author / Version / Date / History
- 2、命名分层可辨：接口按层/子系统前缀(bsp_* / 门面 xxx_* / 算法库前缀)；类型_t；宏全大写
- 3、注释讲"为什么"：尤其数值标度、时序、并发、硬件时刻这类"看代码看不出的理由"
- 4、不跨层、不反向依赖(见 2.3)；关闭功能编译裁剪；魔数上收 config
- 5、通用/可复用代码：注释与标识符不写非开源内部项目名，用通用技术描述替代来源署名(避免 IP 风险)；
开源思想借鉴与本产品自身标识可保留
- 6、本地无工具链时按人工走查对待，交付前务必目标平台 clean rebuild；新增源文件确认已进工程

8. 落地检查清单



复制代码

- 1、目录结构是否"一眼分层"？层名=目录名？
- 2、厂商库/寄存器访问是否只在 bsp+hal？(grep 验证)

- 3、核心算法/业务库是否零硬件依赖、IO 全注入？能否脱离硬件编译？
- 4、每个子系统是否只对外暴露一个门面头？
- 5、随硬件/外设/业务变化的量是否全在 `config? .c` 里有无魔数/引脚/屏参？
- 6、关闭的功能是否真正编译裁剪(省资源、降耦合)？
- 7、裸机/RTOS 是否经统一抽象、业务无 OS API 直调？
- 8、实时/事件任务是否按尺度分层、互不阻塞？共享数据是否单点化、无竞态？
- 9、异常"检测"与"处理"是否解耦？有分级与安全兜底？
- 10、"加一外设/加一功能/加一界面/加一接口"是否可按配方以加法完成？
- 11、文件头/命名/注释是否统一？有无非开源内部项目名残留？
- 12、新增源文件是否已加入工程各目标，交付前是否 `clang -Werror -fsyntax-only` 过？

附. 典型系统的层次映射（同一架构，不同系统）

同一套分层，模块内容随系统类型而变。以下三例：



复制代码

示例 A — 电机 / 运动控制器

`application` : 运动/差速、电流环门面、通信协议、安全
`middleware.lib` : FOC、位置/速度估算、电流重构
`middleware.user` : NTC 温度换算等
`bsp` : PWM、ADC、CAN、编码器、Flash

示例 B — UI / HMI 交互板

`application` : 界面状态机、菜单/页面、输入(按键/触摸)处理、业务编排
`middleware.lib` : GUI 绘制、字库、事件分发、通信协议编解码
`middleware.user` : 显示缓冲/色彩格式适配等
`bsp` : LCD(SPI/并口)、触摸、背光 PWM、按键、UART/USB

示例 C — 多外设采集 / 主控板

`application` : 采样调度、数据融合/上报、设备管理、指令仲裁
`middleware.lib` : 滤波/融合算法、协议栈、状态机
`middleware.user` : 各传感器原始值->工程量换算
`bsp` : 多路 ADC / I2C / SPI 传感器、CAN/UART、GPIO

三者的业务与中间件内容差异极大，但层次划分、依赖方向、解耦手段、可移植/可扩展做法完全相同——这正是本架构方法论"一致适用于各类嵌入式系统"的体现。

本指导为通用嵌入式系统架构方法论；文中示例（含某电机驱动工程）仅为说明用途，具体接口/文件以各自工程当前代码为准。